

PROJECT: Data cleaning utility

AUTHOR: Khoro Avhavihoni Tshivhula

DATE: 19/05/2026

OBJECTIVE:

1. Detect and handle missing values (drop / fill / impute).
2. Fix incorrect dtypes (dates, numbers) and parse dates.
3. Remove duplicates and standardize column names. 4. Output a cleaned dataset and brief cleaninglog.

Step 1: Import libraries

```
In [3]: import pandas as pd
import numpy as np
from datetime import datetime
```

Step 2: Create a sample "dirty" dataset for demonstration

```
In [4]: dirty_data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Bob', 'Alice', 'Frank', None],
    'Age': ['25', 'thirty two', '37', '45', '28', '32', '25', 'forty', '35'],
    'Salary': [50000, '60000', 75000, None, 52000, 60000, 50000, 80000, 72000],
    'Joining Date': ['2021-01-15', '2020-03-22', '2019/07/12', '2022-11-05', '2020-12-01',
                    '2020-03-22', '2021-01-15', '2023-02-28', '2022-09-10'],
    'Department': ['HR', 'IT', 'IT', 'Finance', 'HR', 'IT', 'HR', 'Finance', 'IT'],
    'Email': ['alice@email.com', 'bob@email.com', 'charlie@email.com', 'diana@email.com',
             'eve@email.com', 'bob@email.com', 'alice@email.com', 'frank@email.com', 'john@email.com']
}

df = pd.DataFrame(dirty_data)
print("--- Original (Dirty) DataFrame ---")
```

```
print(df)
print("\n")
```

--- Original (Dirty) DataFrame ---

	Name	Age	Salary	Joining Date	Department	Email
0	Alice	25	50000	2021-01-15	HR	alice@email.com
1	Bob	thirty two	60000	2020-03-22	IT	bob@email.com
2	Charlie	37	75000	2019/07/12	IT	charlie@email.com
3	Diana	45	None	2022-11-05	Finance	diana@email.com
4	Eve	28	52000	2020-12-01	HR	eve@email.com
5	Bob	32	60000	2020-03-22	IT	bob@email.com
6	Alice	25	50000	2021-01-15	HR	alice@email.com
7	Frank	forty	80000	2023-02-28	Finance	frank@email.com
8	None	35	72000	2022-09-10	IT	john@email.com

Step 3. Standardize column names

```
In [5]: df.columns = df.columns.str.lower().str.replace(' ', '_')
print("--- After standardizing column names ---")
print(df.columns.tolist())
print("\n")
```

--- After standardizing column names ---

```
['name', 'age', 'salary', 'joining_date', 'department', 'email']
```

Step 4. Fix incorrect data types

```
In [6]: # a) Age column: contains numbers and text like 'thirty two' -> convert to numeric, invalid becomes NaN
df['age'] = pd.to_numeric(df['age'], errors='coerce')

# b) Salary column: already mix of int and string, convert to numeric
df['salary'] = pd.to_numeric(df['salary'], errors='coerce')
```

```
# c) Joining Date: parse as datetime (handle different formats)
df['joining_date'] = pd.to_datetime(df['joining_date'], errors='coerce')

print("--- After fixing data types (Age as numbers, Salary as numbers, Joining Date as datetime) ---")
print(df.dtypes)
print(df.head(3))
print("\n")
```

--- After fixing data types (Age as numbers, Salary as numbers, Joining Date as datetime) ---

```
name          object
age           float64
salary        float64
joining_date  datetime64[ns]
department    object
email         object
dtype: object
```

	name	age	salary	joining_date	department	email
0	Alice	25.0	50000.0	2021-01-15	HR	alice@email.com
1	Bob	NaN	60000.0	2020-03-22	IT	bob@email.com
2	Charlie	37.0	75000.0	NaT	IT	charlie@email.com

Step 5. Detect and handle missing values

```
In [8]: print("--- Missing value count before handling ---")
print(df.isnull().sum())
print("\n")

# Strategy:
# - Drop rows where 'name' is missing (critical field)
df = df.dropna(subset=['name'])

# - Fill missing 'age' with median age
median_age = df['age'].median()
df['age'].fillna(median_age, inplace=True)

# - Fill missing 'salary' with mean salary
mean_salary = df['salary'].mean()
```

```
df['salary'].fillna(mean_salary, inplace=True)

# - For 'joining_date', we can fill missing with a placeholder (e.g., the earliest date or leave as is)
# Here we fill with the most frequent date (mode) - but for simplicity, we forward fill.
df['joining_date'].ffill(inplace=True)

print("--- Missing value count after handling ---")
print(df.isnull().sum())
print("\n")
```

--- Missing value count before handling ---

```
name          0
age           0
salary        0
joining_date  0
department    0
email         0
dtype: int64
```

--- Missing value count after handling ---

```
name          0
age           0
salary        0
joining_date  0
department    0
email         0
dtype: int64
```

Step 6. Remove duplicate rows

```
In [9]: initial_rows = len(df)
df = df.drop_duplicates()
removed_duplicates = initial_rows - len(df)
print(f"Removed {removed_duplicates} duplicate row(s).")
print("\n")
```

Removed 1 duplicate row(s).

Step 7. Create a brief cleaning log

```
In [10]: cleaning_log = f"""
Cleaning Log
=====
1. Standardized column names: converted to lowercase and replaced spaces with underscores.
2. Fixed data types: Age and Salary converted to numeric (invalid entries became NaN, later imputed).
   Joining Date converted to datetime.
3. Missing values handled:
   - Dropped 1 row where Name was missing.
   - Filled Age missing values with median ({median_age}).
   - Filled Salary missing values with mean ({mean_salary:.2f}).
   - Filled Joining Date missing values using forward fill.
4. Duplicate rows removed: {removed_duplicates} duplicate(s) deleted.
5. Final dataset has {len(df)} rows and {len(df.columns)} columns.
"""

print(cleaning_log)
```

```
Cleaning Log
=====
1. Standardized column names: converted to lowercase and replaced spaces with underscores.
2. Fixed data types: Age and Salary converted to numeric (invalid entries became NaN, later imputed).
   Joining Date converted to datetime.
3. Missing values handled:
   - Dropped 1 row where Name was missing.
   - Filled Age missing values with median (30.0).
   - Filled Salary missing values with mean (61000.00).
   - Filled Joining Date missing values using forward fill.
4. Duplicate rows removed: 1 duplicate(s) deleted.
5. Final dataset has 7 rows and 6 columns.
```

Step 8. Output the cleaned dataset to CSV

```
In [12]: df.to_csv('cleaned_dataset.csv', index=False)
print("Cleaned dataset saved as 'cleaned_dataset.csv'")

# Also save the cleaning log as a text file
with open('cleaning_log.txt', 'w') as f:
    f.write(cleaning_log)
print("Cleaning log saved as 'cleaning_log.txt'")

# Show the cleaned DataFrame
print("\n--- Final Cleaned DataFrame ---")
print(df)
```

Cleaned dataset saved as 'cleaned_dataset.csv'

Cleaning log saved as 'cleaning_log.txt'

--- Final Cleaned DataFrame ---

	name	age	salary	joining_date	department	email
0	Alice	25.0	50000.0	2021-01-15	HR	alice@email.com
1	Bob	30.0	60000.0	2020-03-22	IT	bob@email.com
2	Charlie	37.0	75000.0	2020-03-22	IT	charlie@email.com
3	Diana	45.0	61000.0	2022-11-05	Finance	diana@email.com
4	Eve	28.0	52000.0	2020-12-01	HR	eve@email.com
5	Bob	32.0	60000.0	2020-03-22	IT	bob@email.com
7	Frank	30.0	80000.0	2023-02-28	Finance	frank@email.com

In []: